

ia-26/GA.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 import functions as f
5
6 class Individual:
7
8     def __init__(
9         self,
10         dim,
11         bounds,
12         objective_function
13     ):
14
15         self.objective_function = objective_function
16
17         self.chromosome = np.random.uniform(
18             bounds[0],
19             bounds[1],
20             dim
21         )
22
23         self.fitness = self.objective_function(
24             self.chromosome
25         )
26
27     def evaluate(self):
28
29         self.fitness = self.objective_function(
30             self.chromosome
31         )
32
33
34 # Seleção por torneio
35 def tournament_selection(
36     population,
37     tournament_size=5
38 ):
39
40     candidates = np.random.choice(
41         population,
42         tournament_size
43     )
44
45     best = min(
46         candidates,
47         key=lambda ind: ind.fitness
48     )
49
50     return best
```

```
51
52 # Crossover aritmético
53 def crossover(parent1, parent2):
54
55     alpha = np.random.rand()
56
57     child1 = alpha * parent1.chromosome + \
58         (1 - alpha) * parent2.chromosome
59
60     child2 = alpha * parent2.chromosome + \
61         (1 - alpha) * parent1.chromosome
62
63     return child1, child2
64
65
66 # Mutação gaussiana
67 def mutation(
68     chromosome,
69     bounds,
70     mutation_rate=0.1,
71     mutation_strength=2.0
72 ):
73
74     for i in range(len(chromosome)):
75
76         if np.random.rand() < mutation_rate:
77
78             chromosome[i] += np.random.normal(
79                 0,
80                 mutation_strength
81             )
82
83     chromosome = np.clip(
84         chromosome,
85         bounds[0],
86         bounds[1]
87     )
88
89     return chromosome
90
91
92 # Algoritmo Genético
93 def genetic_algorithm(
94     objective_function,
95     dim=30,
96     population_size=100,
97     generations=1000,
98     bounds=(-100, 100),
99     crossover_rate=0.9,
100    mutation_rate=0.1
101 ):
102
103     # Inicializa população
```

```

104     population = [
105         Individual(
106             dim,
107             bounds,
108             objective_function
109         )
110     ]
111     for _ in range(population_size)
112 ]
113
114 # Melhor indivíduo global
115 best_individual = min(
116     population,
117     key=lambda ind: ind.fitness
118 )
119
120 history = []
121
122 # Loop principal
123 for generation in range(generations):
124     new_population = []
125
126     elite = min(
127         population,
128         key=lambda ind: ind.fitness
129     )
130
131     while len(new_population) < population_size:
132
133         # Seleção
134         parent1 = tournament_selection(
135             population,
136         )
137
138         parent2 = tournament_selection(
139             population
140         )
141
142         # Crossover
143         if np.random.rand() < crossover_rate:
144             child1_chromosome, child2_chromosome = (
145                 crossover(parent1, parent2)
146             )
147
148         else:
149             child1_chromosome = (
150                 parent1.chromosome.copy()
151             )
152
153             child2_chromosome = (
154                 parent2.chromosome.copy()
155             )
156

```

```

157         child2_chromosome = (
158             parent2.chromosome.copy()
159         )
160
161         # Mutação
162         child1_chromosome = mutation(
163             child1_chromosome,
164             bounds,
165             mutation_rate
166         )
167
168         child2_chromosome = mutation(
169             child2_chromosome,
170             bounds,
171             mutation_rate
172         )
173
174         # Cria filhos
175         child1 = Individual(
176             dim,
177             bounds,
178             objective_function
179         )
180
181         child2 = Individual(
182             dim,
183             bounds,
184             objective_function
185         )
186
187         child1.chromosome = child1_chromosome
188         child2.chromosome = child2_chromosome
189
190         child1.evaluate()
191         child2.evaluate()
192
193         new_population.append(child1)
194         new_population.append(child2)
195
196     #ELITISMO
197     new_population[0] = elite
198
199     # Atualiza população
200     population = new_population[:population_size]
201
202     # Melhor indivíduo da geração
203     current_best = min(
204         population,
205         key=lambda ind: ind.fitness
206     )
207
208     if current_best.fitness < best_individual.fitness:
209

```

```

210         best_individual = current_best
211
212     history.append(best_individual.fitness)
213
214     print(
215         f"Geração {generation+1:4d}"
216         f" | Melhor valor: "
217         f"{best_individual.fitness:.10f}"
218     )
219
220     return (
221         best_individual.chromosome,
222         best_individual.fitness,
223         history
224     )
225
226
227 def plot_convergence(history):
228
229     plt.figure(figsize=(8, 5))
230
231     plt.plot(history)
232
233     plt.title("Convergência do GA")
234
235     plt.xlabel("Geração")
236     plt.ylabel("Melhor valor")
237
238     plt.yscale("log")
239
240     plt.grid(True)
241
242     plt.show()
243
244
245 if __name__ == "__main__":
246
247     objective_function = f.f1
248
249     bounds = (-100, 100)
250
251     best_position, best_value, history = (
252         genetic_algorithm(
253             objective_function=objective_function,
254             dim=30,
255             population_size=50,
256             generations=3000,
257             bounds=bounds,
258             crossover_rate=0.9,
259             mutation_rate=0.1
260         )
261     )
262

```

```

263     plot_convergence(history)
264
265     print("\n===== RESULTADO FINAL =====")
266
267     print("Melhor posição:")
268     print(best_position)
269
270     print("\nMelhor valor:")
271     print(best_value)

```